

Chapter 4: The Processor

Complete Lecture Notes (Sections 4.5–4.8)

Computer Architecture Course

Contents

Part I

Section 4.5: An Overview of Pipelining

1 Introduction to Pipelining

Pipelining is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

- **Core Concept:** Just as a car assembly line works on multiple cars simultaneously, a pipelined processor works on different stages of multiple instructions at the same time.
- **Key Insight:** It improves **throughput** (instructions per unit time), not latency of individual instructions.

2 The Classic Five-Stage RISC-V Pipeline

1. **IF (Instruction Fetch):** Fetch instruction from memory using PC.
2. **ID (Instruction Decode):** Decode instruction and read registers.
3. **EX (Execution):** Perform ALU operation or address calculation.
4. **MEM (Memory Access):** Read/Write data memory.
5. **WB (Write Back):** Write result to register file.

3 Pipeline Performance

- **Single-Cycle Clock:** Must be long enough for slowest instruction (e.g., 800ps for load).
- **Pipelined Clock:** Must be long enough for the slowest *stage* (e.g., 200ps).
- **Theoretical Speedup:** Pipeline depth (5x for a 5-stage pipeline) in ideal conditions.

4 Design for Pipelining

Why is RISC-V good for pipelining?

- **Fixed-length instructions (32-bit):** IF and ID stages are simple; next instruction is at PC+4.
- **Few instruction formats:** Decoding is simpler.
- **Load-Store architecture:** Memory access only in load/store instructions.

5 Pipeline Hazards

A **hazard** is a situation where the next instruction cannot execute in the following clock cycle.

5.1 1. Structural Hazards

Problem: Two instructions need the same hardware resource.

Solution: Separate Instruction Memory and Data Memory (or caches).

5.2 2. Data Hazards

Problem: An instruction needs data that hasn't been written back yet.

Solutions:

- **Forwarding (Bypassing):** Send result directly from ALU to next instruction.
- **Stalling:** Insert bubbles when forwarding is impossible (load-use hazard).

5.3 3. Control Hazards

Problem: The processor doesn't know which instruction to fetch next (branches).

Solutions: Branch prediction, delayed branches, stalling.

Part II

Section 4.6: Pipelined Datapath and Control

6 Pipeline Registers

To share a single datapath among multiple instructions, we place **Pipeline Registers** between each stage:

- **IF/ID (96 bits):** Holds 32-bit Instruction + 64-bit PC.
- **ID/EX (256 bits):** Holds Register Values, Immediate, PC, etc.
- **EX/MEM (193 bits):** Holds ALU Result, Write Data, Destination Register.
- **MEM/WB (128 bits):** Holds Memory Data, ALU Result, Destination Register.

No Pipeline Register at End

There is no pipeline register after WB because the state (Register File, Memory, PC) is updated directly.

7 Instruction Walkthroughs

7.1 Load Instruction (ld)

- **IF:** Fetch instruction, increment PC. Save to IF/ID.
- **ID:** Read registers, sign-extend immediate. Save to ID/EX.
- **EX:** ALU adds register + immediate. Save address to EX/MEM.
- **MEM:** Read data memory. Save to MEM/WB.
- **WB:** Write loaded data into Register File.

7.2 Store Instruction (sd)

Same as load, except:

- **MEM:** Write data to memory.
- **WB:** **Nothing happens** (no register write).

8 The Destination Register Bug

Critical Design Bug

The destination register is decoded in **ID**, but the write occurs in **WB**. By then, a *different* instruction is in ID!

The Fix: Pass the destination register number (**rd**) through all pipeline registers (ID/EX → EX/MEM → MEM/WB).

9 Pipelined Control

Control signals are generated in the **ID stage** but used later:

- **EX Stage:** ALUOp, ALUSrc
- **MEM Stage:** Branch, MemRead, MemWrite
- **WB Stage:** MemtoReg, RegWrite

Control signals are passed through pipeline registers to arrive at the correct stage.

Part III

Section 4.7: Data Hazards: Forwarding versus Stalling

10 Data Hazard Example

Consider this instruction sequence:

```
sub x2, x1, x3    // Writes x2
and x12, x2, x5   // Reads x2 - HAZARD
or  x13, x6, x2   // Reads x2 - HAZARD
add x14, x2, x2   // Reads x2 - HAZARD
sd  x15, 100(x2)  // Reads x2 - No Hazard (x2 is ready)
```

The `and` and `or` instructions read `x2` before `sub` writes it back!

11 Forwarding (Bypassing)

Solution

Forward the result directly from the ALU output (or memory) to the instruction that needs it, *before* it is written to the register file.

11.1 Hazard Conditions

- 1a. `EX/MEM.RegisterRd = ID/EX.RegisterRs1` (Forward to first operand)
- 1b. `EX/MEM.RegisterRd = ID/EX.RegisterRs2` (Forward to second operand)
- 2a. `MEM/WB.RegisterRd = ID/EX.RegisterRs1` (Forward older value)
- 2b. `MEM/WB.RegisterRd = ID/EX.RegisterRs2` (Forward older value)

11.2 Forwarding Control Values

ForwardA/B	Source
00	ID/EX (no forwarding)
10	EX/MEM (prior ALU result)
01	MEM/WB (older result or memory data)

11.3 Double Data Hazard

When two instructions both produce a result (e.g., `add x1,...; add x1,...; add x1,x1,...`), forward from the **most recent** one. MEM hazard condition must check: NOT a more recent EX hazard.

12 Data Hazards and Stalls

Load-Use Hazard

When a load is immediately followed by an instruction that uses its result, forwarding **cannot help**. The data isn't available until after the MEM stage!

Solution: The **Hazard Detection Unit** inserts a **bubble** (stall) for one cycle:

```
if (ID/EX.MemRead
    and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
        or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    Stall the pipeline
```

Part IV

Section 4.8: Control Hazards

13 The Problem

A **control hazard** (branch hazard) is a delay in determining the proper instruction to fetch. In our basic design, the branch decision occurs in the **MEM stage**, meaning 3 incorrect instructions may have been fetched.

14 Scheme 1: Assume Branch Not Taken

Predict that the branch is NOT taken, continue sequential execution. If wrong, **flush** the incorrectly fetched instructions (set control signals to 0).

15 Reducing Branch Delay

Move branch decision to the **ID stage**:

- Move branch adder from EX to ID.
- Add equality test unit in ID.
- **Result:** Only 1 instruction needs to be flushed (instead of 3).

New Stall Requirements

- ALU instruction before branch → 1-cycle stall
- Load instruction before branch → 2-cycle stall

16 Dynamic Branch Prediction

Branch Prediction Buffer

A small memory indexed by the lower portion of the branch address, containing bits indicating if the branch was recently taken.

16.1 1-Bit vs 2-Bit Prediction

- **1-Bit:** Mispredicts twice per loop (first and last iteration).
- **2-Bit:** Must be wrong twice before prediction changes. Better for loops!

16.2 Advanced Predictors

- **Branch Target Buffer:** Caches the target address or instruction.
- **Correlating Predictor:** Combines local and global branch behavior.
- **Tournament Predictor:** Uses multiple predictors and a selector.

Part V

Chapter 4 Exercises with Answers

Note on Answering Exercises

Each answer references the relevant section where the topic was covered in our lecture notes.

17 Exercise 4.1: AND Instruction Analysis

Instruction: `and rd, rs1, rs2`

Interpretation: $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] \text{ AND } \text{Reg}[\text{rs2}]$

4.1.1 Control Signal Values

Answer (Section 4.6: Pipelined Control)

For an R-type instruction like `and`:

- **RegWrite** = 1 (writes to register file)
- **ALUSrc** = 0 (second operand from register, not immediate)
- **MemtoReg** = 0 (result from ALU, not memory)
- **MemRead** = 0 (no memory read)
- **MemWrite** = 0 (no memory write)
- **Branch** = 0 (not a branch)
- **ALUOp** = 10 (R-type operation)

4.1.2 Useful Resources

Answer (Section 4.6: Pipelined Datapath)

- **Instruction Memory:** Fetches the instruction
- **Register File:** Reads `rs1` and `rs2`, writes `rd`
- **ALU:** Performs the AND operation
- **Control Unit:** Generates control signals

4.1.3 Unused Resources

Answer (Section 4.6)

- **Data Memory:** Produces no output (not accessed)
- **Immediate Generator:** Produces output but it's not used
- **Branch Adder:** Produces output but it's not used

18 Exercise 4.3: Instruction Mix Analysis

4.3.1 Fraction Using Data Memory

Answer (Section 4.5-4.6)

Only `load` and `store` instructions use data memory. Typical mix: 20-25% of instructions.

4.3.2 Fraction Using Instruction Memory

Answer (Section 4.5)

100% - Every instruction must be fetched from instruction memory.

4.3.3 Fraction Using Sign Extend

Answer (Section 4.6)

Loads, stores, branches, and I-type instructions use sign extend. Approximately 50-60%.

4.3.4 Sign Extend During Unused Cycles

Answer (Section 4.6)

The sign extend unit produces output every cycle regardless, but the output is simply **ignored** by subsequent multiplexors when not needed.

19 Exercise 4.4: Stuck-at-0 Faults

4.4.1 MemToReg Stuck at 0

Answer (Section 4.6: Pipelined Control)

Load instructions fail. MemToReg=0 selects ALU result instead of memory data. The loaded value would be incorrect (would get the address calculation result instead of the loaded data).

4.4.2 ALUSrc Stuck at 0

Answer (Section 4.6)

I-type instructions, loads, and stores fail. ALUSrc=0 means the second ALU input always comes from the register file instead of the immediate. Address calculations would be wrong.

20 Exercise 4.7: Latency Calculations

4.7.1 R-type Latency

Answer (Section 4.5-4.6)

R-type critical path: $IF + ID + EX + WB = PC\ read + IMem + RegRead + ALU + RegWrite$. Using typical values: $30 + 250 + 150 + 200 + 20 = \mathbf{650ps}$.

4.7.2 Load (ld) Latency

Answer (Section 4.6)

Load is the longest: $IF + ID + EX + MEM + WB$. $PC\ read + IMem + RegRead + Mux + ALU + DMem + Mux + RegWrite = \mathbf{950-1000ps}$ typically.

4.7.6 Minimum Clock Period

Answer (Section 4.5)

The minimum clock period equals the **latency of the slowest instruction** (load), divided by the number of pipeline stages for pipelined design, or the full latency for single-cycle.

21 Exercise 4.16: Pipelining Performance

4.16.1 Clock Cycle Time

Answer (Section 4.5)

- **Non-pipelined:** Sum of all stage latencies (entire instruction)
- **Pipelined:** Maximum of individual stage latencies + pipeline latch overhead

4.16.4 Data Memory Utilization

Answer (Section 4.5-4.6)

Data memory is used only by load and store instructions. Utilization = fraction of loads + stores in instruction mix (typically 20-30%).

22 Exercise 4.18-4.20: Data Hazard Examples

4.18 Without Hazard Handling

Answer (Section 4.7: Data Hazards)

Given:

```
addi x11, x12, 5
add  x13, x11, x12
addi x14, x11, 15
```

Without forwarding: x11 is not written until WB of first instruction. Second instruction reads **old value** of x11 (=11). $x13 = 11 + 22 = 33$. Third instruction also reads old x11 (=11). $x14 = 11 + 15 = 26$.

4.20 Adding NOPs for Correctness

Answer (Section 4.7)

```
addi x11, x12, 5
nop
nop
add  x13, x11, x12
addi x14, x11, 15
nop
nop
add  x15, x13, x12
```

Two NOPs are needed after each producer to allow WB before the consumer's ID.

23 Exercise 4.24: Hazard Detection Unit

Answer (Section 4.7: Stalling)

Choice 2 is correct:

```
ld x11, 0(x12):    IF ID EX ME WB
add x13, x11, x14:  IF ID..EX ME WB
or x15, x16, x17:   IF..ID EX ME WB
```

The hazard detection unit detects the load-use hazard during ID of the `add`. It stalls the `add` in ID (ID..) and freezes IF (IF..), inserting a bubble into EX.

24 Exercise 4.28: Branch Prediction

4.28.1 CPI with Always-Taken Predictor

Answer (Section 4.8: Branch Prediction)

If branches are taken 45% of the time:

- Misprediction rate = 55% (not taken when we predicted taken)
- Extra CPI = (branch frequency) $\times$ (mispredict rate) $\times$ (penalty)
- Extra CPI = $0.25 \times 0.55 \times 1 = \mathbf{0.1375}$ cycles

4.28.3 CPI with 2-Bit Predictor

Answer (Section 4.8)

With 85% accuracy:

- Misprediction rate = 15%
- Extra CPI = $0.25 \times 0.15 \times 1 = \mathbf{0.0375}$ cycles

25 Exercise 4.29: Branch Prediction Accuracy

Pattern: T, NT, T, T, NT

4.29.1 Static Predictor Accuracy

Answer (Section 4.8)

- **Always-Taken:** 3 correct (T, T, T), 2 wrong (NT, NT) = **60%**
- **Always-Not-Taken:** 2 correct (NT, NT), 3 wrong (T, T, T) = **40%**

4.29.3 2-Bit Predictor Accuracy (Repeated Forever)

Answer (Section 4.8: 2-Bit Prediction)

After warmup, the 2-bit predictor settles into a pattern. For T, NT, T, T, NT repeated:

- Starting in Strongly Taken: T(correct), NT(wrong→WT), T(correct), T(correct→ST), NT(wrong→WT)
- Pattern repeats with 3/5 correct = **60%**

26 Check Yourself Answers (From Textbook)

§4.5, page 275

1. Stall due to a load-use data hazard of the ld result.
2. Avoid stalling by forwarding the add result.
3. It need not stall, even without forwarding.

§4.6, page 288

Statements 2 and 4 are correct; the rest are incorrect.

§4.8, page 314

1. Predict not taken.
2. Predict taken.
3. Dynamic prediction.